Danmarks
Tekniske
Universitet

Lecture 2

Microcontroller Development for Advanced IoT using Embedded C (34373)

Author

Hákon Hlynsson - s225765

February 15, 2026

Contents

1 Execise 1 **1**

2 Execise 2 **5**

3 Execise 3 **7**

4 Execise 4 **9**

References **11**

5 Appendix **12**

1 Execise 1

a) Why use both crystal and RC-oscillators?

It gives the user the ability to choose the clock that best matches the project, depending on what they are looking for in the clock, whether it be accuracy, cost, size, power consumption, or reliability.

b) Advantages/disadvantages of using crystal?

The advantages of using a crystal are that you get a cleaner clock that can operate at higher frequencies, but the disadvantages are that it is more expensive, takes up more space, and uses more power.

c) Advantages/disadvantages of using RC-oscillator?

The advantages of using an RC-oscillator are that it is integrated into the chip, therefore making it cheap, small, and power efficient. The disadvantages are that it does not create a clean clock and does not support as high frequencies.

d) Which crystals oscillators (frq) do we have available on Nucleo-F302R8?

The Nucleo-F302R8 comes equipped with two crystal oscillators a HSE (High Speed External) and an LSE (Low Speed External). The HSE operates at 8 MHz, and the LSE operates at 32.768 kHz (2^{15} Hz).

e) Explain what is the Low-Speed External clock (LSE) and Low-Speed Internal clock (LSI). What are the differences, how are they each initialized and what are the frequencies?

The biggest difference between the LSI (Low Speed Internal) and the LSE (Low Speed External) is that the LSI is an RC oscillator, while the LSE is a crystal oscillator. Another difference is that the LSE operates at 32.768 kHz (2^{15} Hz), whereas the LSI operates at 40 kHz.

The way you initialize the LSI and LSE is by opening STM32CubeMX and selecting your desired board. After that, go to Pinout & Configuration, then open System Core and select RCC (Reset and Clock Control). This opens a configuration window where you can select the low speed clock using the dropdown menu. In the menu, you can choose from the following options:

- **Disable:** Uses the internal clock LSI (40 kHz).
- **BYPASS Clock Source:** Uses a clock signal coming from another chip, such as the ST-LINK or another MCU.
- **Crystal/Ceramic Resonator:** Uses the external crystal clock LSE (32.768 kHz).

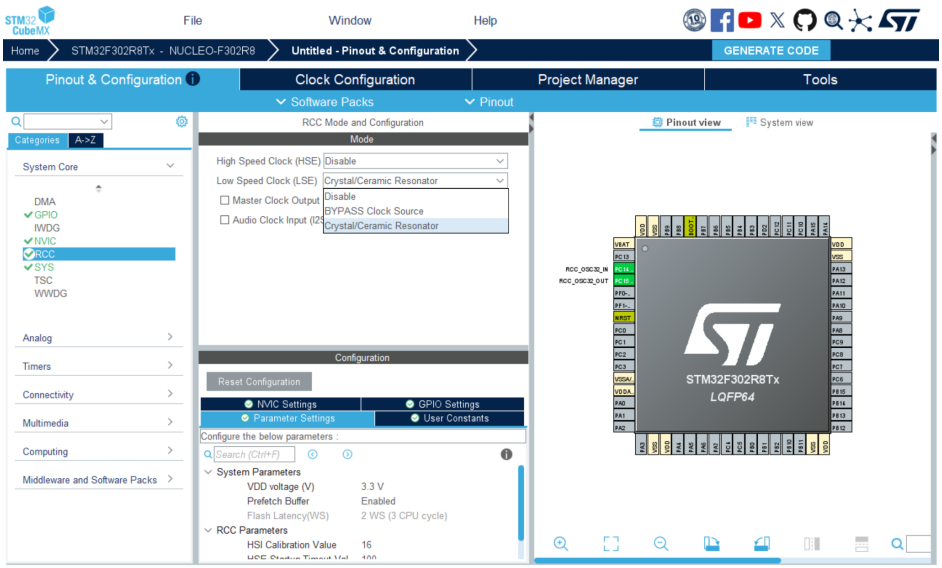


Figure 1: Where to enableing the LSI and LSE clock

f) Why do we have both HSE and LSE clocks?

The reason that we have both an HSE (High Speed External) and an LSE (Low Speed External) clock is because it gives the user flexibility to choose the one that best matches their project. It is preferable to use the HSE when high speed communication such as SPI is needed. Meanwhile, if a precise low speed clock is needed for timekeeping, the LSE is more optimal.

g) What is the duty-cycle typically of a clock signal?

In a typical clock signal, the duty cycle is 50%, meaning that during a period it spends equal time in the low state and the high state.

h) How is the clock period related to frequency?

The clock period is directly related to the frequency through the definition of frequency, since frequency is defined as the number of events per second, and in the case of clocks that is periods. The equation that describes the mathematical relationship is

$$f = \frac{1}{T_p}, \quad \text{where } f = \text{ frequency and } T_p = \text{ clock time period.}$$

i) How could we upscale the crystal sourced 8 MHz to 64 MHz?

To upscale a frequency from 8 MHz to the desired 64 MHz, the STM32F302R8 uses a PLL (Phase-Locked Loop). This is because the PLL has the ability to take an input frequency and generate a multiple of that frequency, either higher or lower. Therefore, to generate 64 MHz, the PLL must be configured with a multiplication factor of 8. The picture below shows a possible configuration setup in STM32CubeMX.

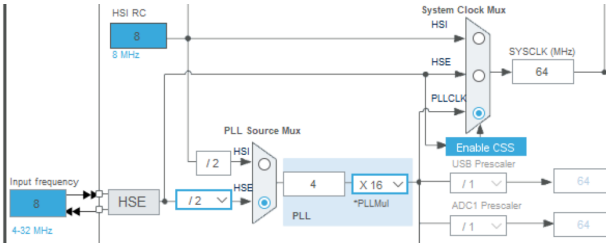


Figure 2: Configuration of a crystal sourced 8 MHz to 64 MHz

j) What is the highest achievable clock frequency on STM32F302?

According to the datasheet, the maximum available clock speed that the STM32F302R8 can run at is 72 MHz. This is achieved by using an 8 MHz input signal and the PLL.

k) What is the Clock Security System (CSS) used for?

The CSS (Clock Security System) monitors the HSE (High-Speed External) clock, and if it detects any failures, it automatically switches to the HSI (High Speed Internal) clock. This allows the program to keep running in case of HSE clock failures.

l) When configuring the high-speed clock, is it better to use internal (HSI) or external (HSE) clock?

It all depends on the overall goal and task of the clocks. If precision and reliability are necessary, then the HSE should be used. However, if power efficiency and small size are the limiting factors, then go with the HSI clock.

m) How many Master clock outputs (MCO) does STM32F411 have?

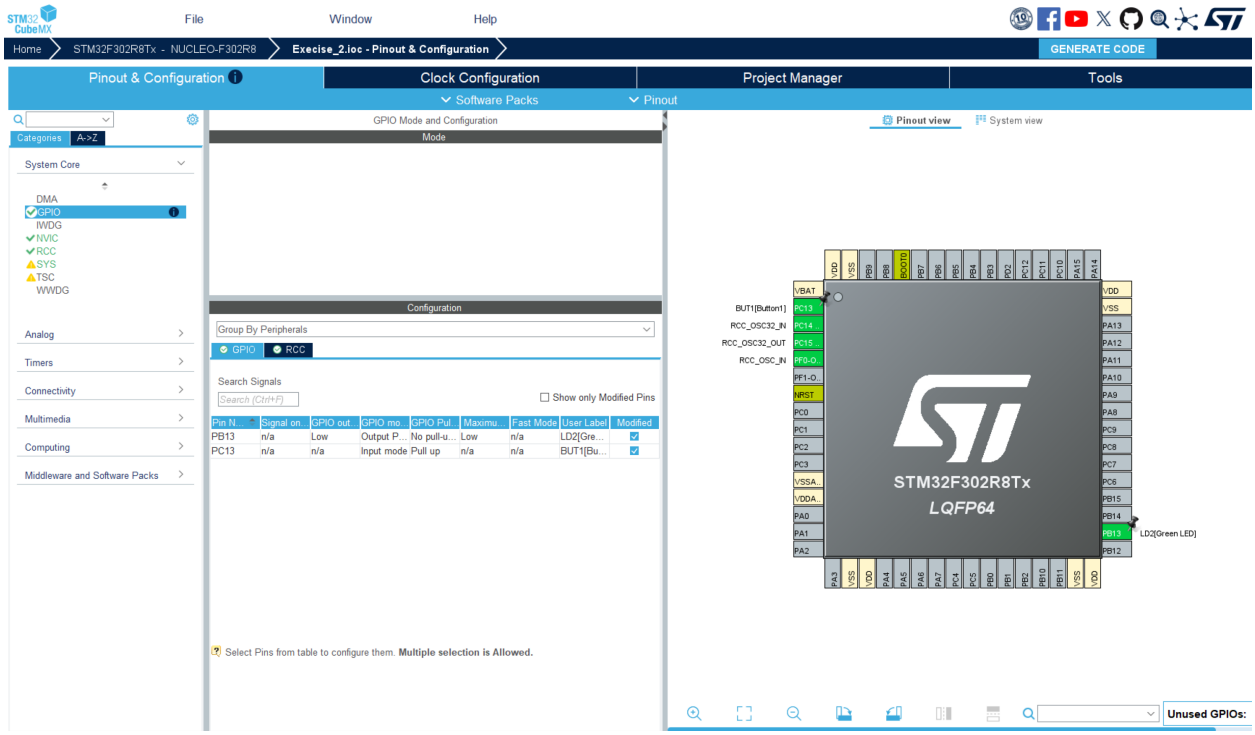
The STM32F411 consists of two MCOs (Master Clock Outputs), called MCO1 and MCO2

n) What can we use MCOs (Master Clock Output) for? How many does STM32F302 have

The MCO gives the board the ability to output its clock on one of its GPIO pins. This can be useful for synchronizing clocks with other MCUs, providing an external clock source for devices, and debugging the clock configuration. The STM32F302R8 consists of one MCO (Master Clock Output), called MCO.

2 Execise 2

Implement button press for LED on/off switching. The pin used for the LED is PB13, and the button uses PC13. You can see the setup in STM32CubeMX below.



The code can be seen below and is also available on GitHub at this link: GitHub repository

Listing 1: EX2: Implement button press for LED on/off switching(Main.c)

```

int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  while (1)
  {
  if (HAL_GPIO_ReadPin(BUT1_GPIO_Port,BUT1_Pin)==GPIO_PIN_RESET ){
      HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
      }
  else{
      HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
      }
  }
}
```

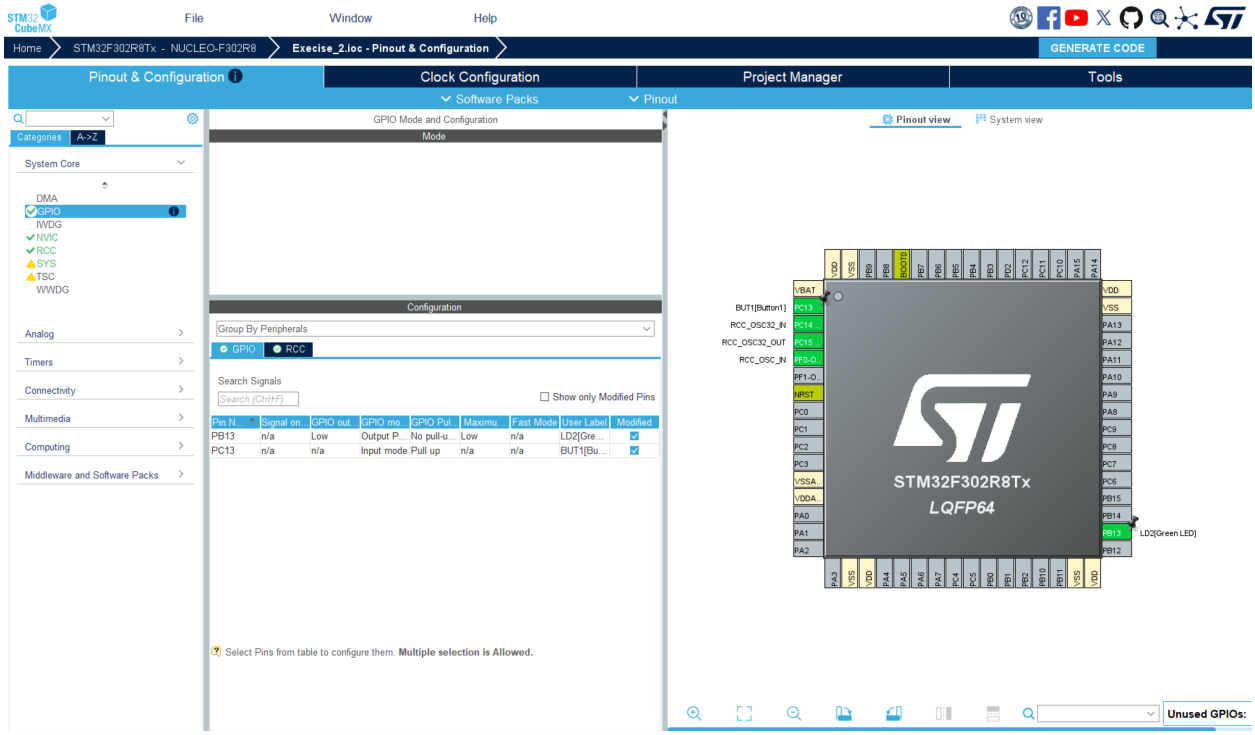
Listing 2: EX2: Implement button press for LED on/off switching(GPIO.c)

```c
#define BUT1_Pin GPIO_PIN_13
#define BUT1_GPIO_Port GPIOC
#define LD2_Pin GPIO_PIN_13
#define LD2_GPIO_Port GPIOB

void MX_GPIO_Init(void)
{

  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOF_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);

  /*Configure GPIO pin : BUT1_Pin */
  GPIO_InitStruct.Pin = BUT1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(BUT1_GPIO_Port, &GPIO_InitStruct);

  /*Configure GPIO pin : LD2_Pin */
  GPIO_InitStruct.Pin = LD2_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
}
```

Link to video demonstration: https://www.youtube.com/watch?v=0e82dI_e4dE&list=PLqZu0cu4riNv2rUXMMD5VIgopG7LxmTGl&index=1

3 Execise 3

Dual rate blink using button interrup.The pin used for the LED is PB13, and the button uses PC13. You can see the setup in STM32CubeMX below.



The code can be seen below and is also available on GitHub at this link: GitHub repository

Listing 3: EX3: Dual rate blink using button interrup(Main.c)

```

uint16_t tDelay= 250;
int main(void)
{

  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_NVIC_Init();
  while (1)
  {
          HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
          HAL_Delay(tDelay);
  }
}
```

Listing 4: EX3: Dual rate blink using button interrup(GPIO.c)

```c
void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOF_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);

  /*Configure GPIO pin : B1_Pin */
  GPIO_InitStruct.Pin = B1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);

  /*Configure GPIO pin : LD2_Pin */
  GPIO_InitStruct.Pin = LD2_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
}
```

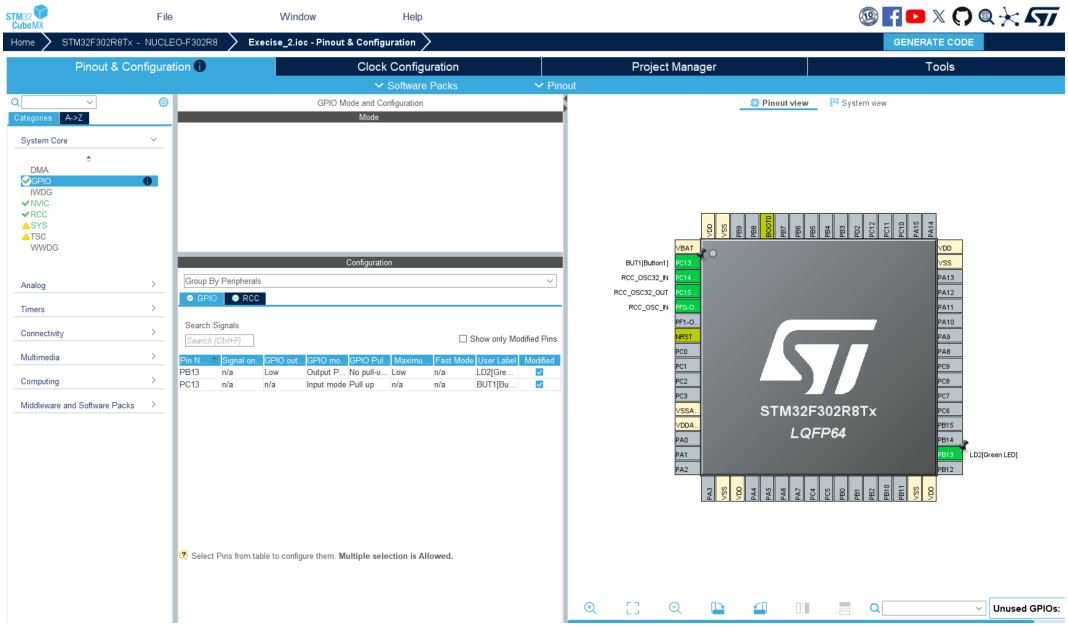
Listing 5: EX3: Dual rate blink using button interrup(stm32f3xx.c)

```c
extern uint16_t tDelay;
void EXTI15_10_IRQHandler(void)
{
        if(HAL_GPIO_ReadPin(B1_GPIO_Port,B1_Pin)){
                if (tDelay==250){
                        tDelay=1000;
                }
                else{
                        tDelay=250;
                }
        }
  HAL_GPIO_EXTI_IRQHandler(B1_Pin);
}
```

Link to video demonstration: `https://www.youtube.com/watch?v=nrLQZ25FEn0&list=PLqZu0cu4riNv2rUXMMD5VIgopG7LxmTGl&index=3`

4 Execise 4

This exercise implements LED on/off switching using an external interrupt. The LED is connected to pin PB13, while the push button is connected to pin PC13. The GPIO configuration in STM32CubeMX is shown below.



The code can be seen below and is also available on GitHub at this link: GitHub repository

Listing 6: EX4: LED on/off switching using an external interrupt(Main.c)

```

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  if (GPIO_Pin == BUT1_Pin)
  {
          HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
  }
}

int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_NVIC_Init();

  while (1)
  {}
}
```

Listing 7: EX4: LED on/off switching using an external interrupt(GPIO.c)

```c

void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOF_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);

  /*Configure GPIO pin : BUT1_Pin */
  GPIO_InitStruct.Pin = BUT1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(BUT1_GPIO_Port, &GPIO_InitStruct);

  /*Configure GPIO pin : LD2_Pin */
  GPIO_InitStruct.Pin = LD2_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
}
```

Listing 8: EX4: LED on/off switching using an external interrupt(stm32f3xx.c.c)

```c
void EXTI15_10_IRQHandler(void)
{
  HAL_GPIO_EXTI_IRQHandler(BUT1_Pin);
}
```

Listing 9: EX4: LED on/off switching using an external interrupt (stm32f3xx_hal_gpio.c)

```c
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}
```

Link to video demonstration: `https://www.youtube.com/watch?v=9EiAdxFlKGQ&list=PLqZu0cu4riNv2rUXMMD5VIgopG7LxmTGl&index=3`

References

5 Appendix